COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	Sep 9, 2026	**DATE SUBMITTED:**	Sep 16, 2026
NAME:	Antonio, Christine Mae P.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 8.1 **Sorting Algorithms Pt2**

1. PROCEDURE OUTPUT

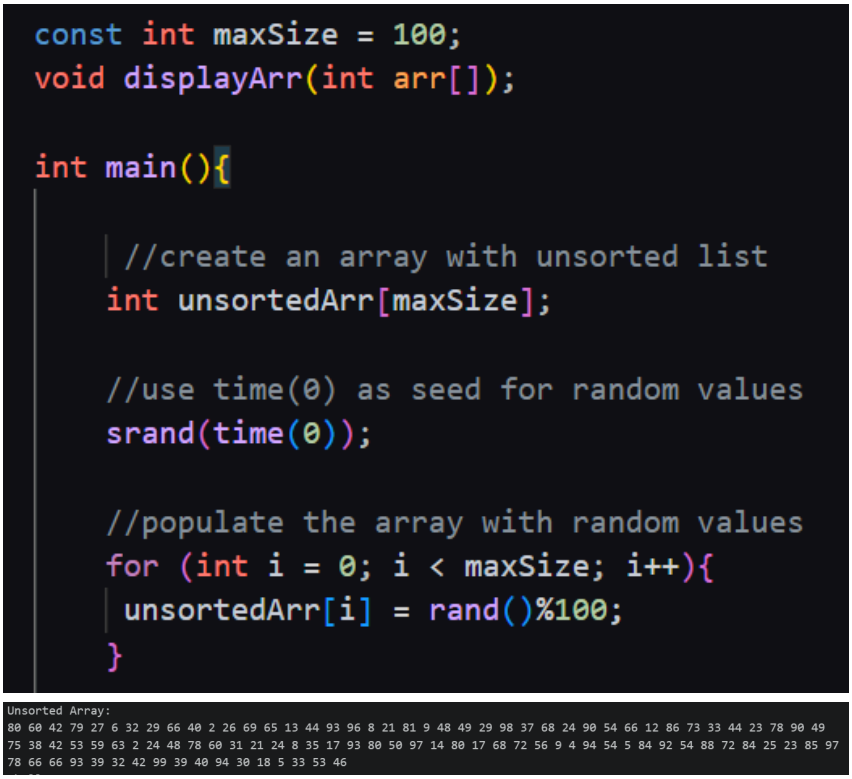
Code + Console Screenshot	
Observations	This creates an unsorted array that has 100 maximum capacity. Its elements are generated by using srand(time(0)) and limiting each element to only be until 100.

Table 8-1. Array of Values for Sort Algorithm Testing

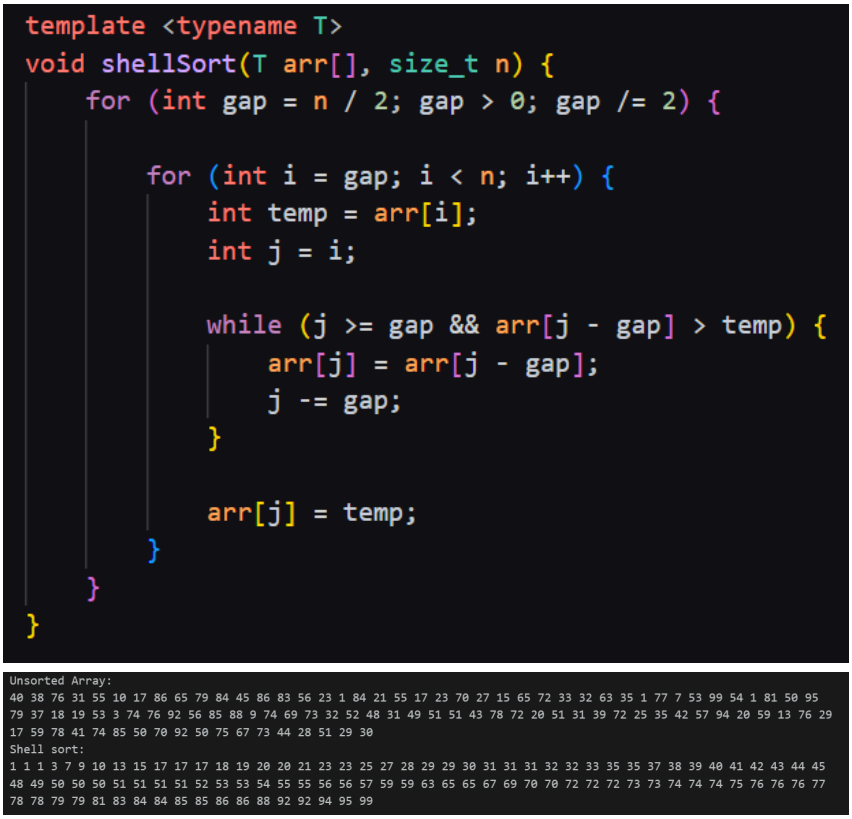
Code + Console Screenshot	
Observations	The shell sort uses decreasing gap sizes. It sorts the elements in smaller groups first, then reduces the gap until it becomes 1, making the final pass similar to insertion sort.

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```
template <typename T>
void mergeSort(T arr[], int left, int mid, int right) {

    int n1 = mid-left + 1;
    int n2 = right - mid;

    int* Left = new int[n1];
    int* Right = new int[n2];

    for (int i = 0; i < n1; i++)
        Left[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        Right[j] = arr[mid + 1 + j];

    int i = 0, j = 0;
    int k = left;

    while (i < n1 && j < n2)
    {
        if (Left[i] <= Right[j]) arr[k++] = Left[i++];
        else arr[k++] = Right[j++];
    }

    while (i < n1) arr[k++] = Left[i++];

    while (j < n2) arr[k++] = Right[j++];

    delete[] Left;
    delete[] Right;
}
```

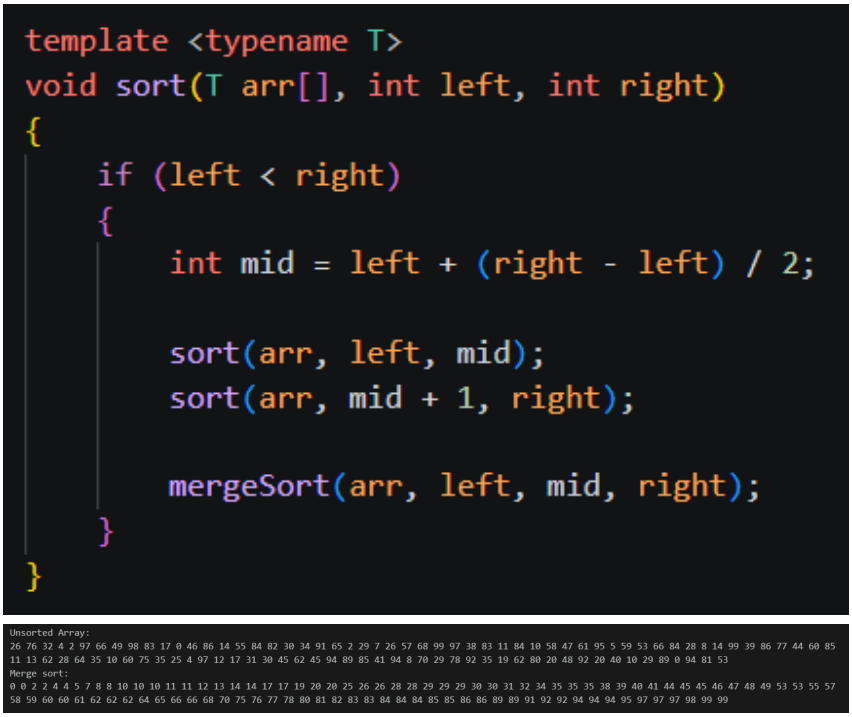
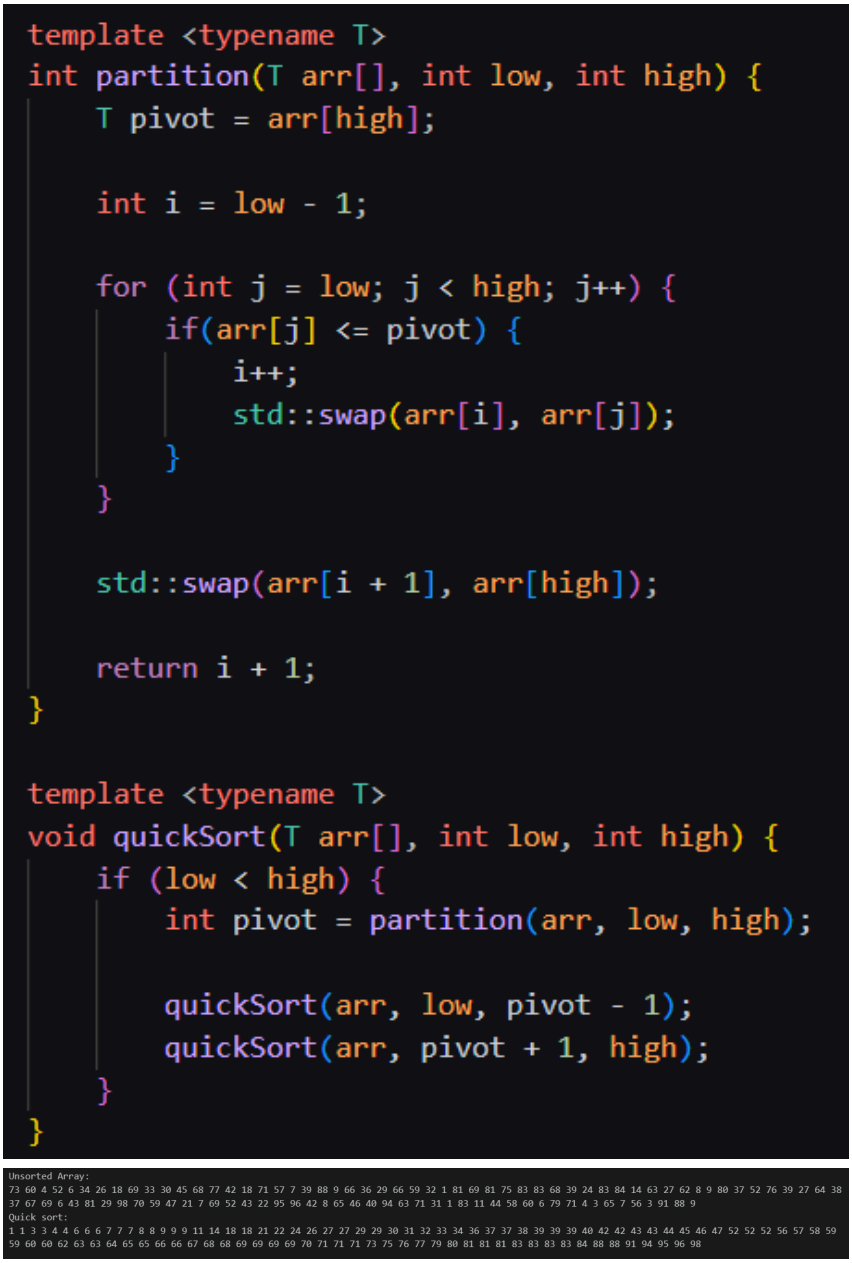
	
Observations	The sort() function repeatedly divides the array into smaller halves until each part contains one element, while mergeSort() compares the elements from the left and right sub lists and merges them together.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	
Observations	The code implements Quick Sort using a partitioning method with the last element as the pivot. The partition() function compares each element with the pivot and moves smaller or equal values to the left, then places the pivot in its correct position. The quickSort() function then recursively sorts the elements on both sides of the pivot until the entire array is sorted.

```
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high];

    int i = low - 1;

    for (int j = low; j < high; j++) {
        if(arr[j] <= pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }

    std::swap(arr[i + 1], arr[high]);

    return i + 1;
}

template <typename T>
void quickSort(T arr[], int low, int high) {
    if (low < high) {
        int pivot = partition(arr, low, high);

        quickSort(arr, low, pivot - 1);
        quickSort(arr, pivot + 1, high);
    }
}
```

```
Unsorted Array:
73 60 4 52 6 34 26 18 69 33 30 45 68 77 42 18 71 57 7 39 88 9 66 36 29 66 59 32 1 81 69 81 75 83 83 68 39 24 83 84 14 63 27 62 8 9 80 37 52 76 39 27 64 38
37 67 69 6 43 81 29 98 70 59 47 21 7 69 52 43 22 95 96 42 8 65 46 40 94 63 71 31 1 83 11 44 58 60 6 79 71 4 3 65 7 56 3 91 88 9
Quick sort:
1 1 3 3 4 4 6 6 6 7 7 7 8 8 9 9 9 11 14 18 18 21 22 24 26 27 27 29 29 30 31 32 33 34 36 37 37 38 39 39 39 40 42 42 43 43 44 45 46 47 52 52 52 56 57 58 59
59 60 60 62 63 63 64 65 65 66 66 67 68 68 69 69 69 70 71 71 71 73 75 76 77 79 80 81 81 81 83 83 83 83 84 88 88 91 94 95 96 98
```

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other

sorting algorithms? Demonstrate an example.

- It is possible to use other sorting algorithms for the partition in quick sort as it only requires the sub lists to be parted into the left side where elements < pivot, and the right side where elements > pivot. For example, insertion sort can be used to sort the sub lists.
  Array: {3, 9, 7, 2, 1}
  Set pivot to the last element, which is 1
  Left side: {3, 2}
  Right side: {9, 7}
  Now, the array is: {3, 2, 1, 9, 7}

  Using insertion sort to sort the left sub list and right sub list:
  Left sub list {3, 2}: {3} → insert 2→ {2, 3}
  Right sub list {9, 7): {9} → insert 7 → {7, 9}

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

- The fastest sorting algorithm is the quick sort with its time complexity of O(n log n). Merge sort and quick sort both uses the divide and conquer algorithm, requiring about log N levels, while processing about N elements at each level.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Expanding the pseudo-codes**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **It was prompted to transform the given into more informative pseudo-codes**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.